ShelfFlow: Local-First Agentic AI Decision Support System

New MVP Proposal for the Final-Year Project

Project type: Agentic AI decision-support upgrade for an existing supermarket stock workflow
Primary users: Supermarket store managers and inventory staff
Pilot scope: One organization, one or two stores, and one perishable product category
Development strategy: Build and evaluate locally first; migrate selected components to AWS only when funding or university support is available.

1. Project idea

ShelfFlow does not replace a supermarket's POS, ERP or inventory system. It accepts structured CSV exports containing products, stores, batches and sales history, then adds an Agentic AI layer that helps managers investigate expiry risk and evaluate possible actions.

The system combines batch-level expiry analysis with a manager-configured 2D store heatmap. A manager can ask which batches need attention, inspect the evidence, simulate moving a product to another shelf position and approve or reject the resulting recommendation. The agent may analyze, calculate and propose, but it must not execute an irreversible operational action without manager approval.

The project contribution is therefore not "another inventory website." It is a controlled Agentic AI decision-support workflow that converts existing stock data into explainable expiry-risk and store-layout recommendations.

2. Problem statement

Supermarket data may already exist in spreadsheets or operational systems, but managers still need to connect batch quantity, expiry date, sales history, store demand and product location when deciding how to handle stock that may expire before being sold. A conventional dashboard can display these values, but it may not explain which issue deserves attention first or allow the manager to compare possible interventions.

ShelfFlow addresses this decision gap. It does **not** guarantee that products will not expire. Instead, it aims to improve the speed, transparency and consistency of decisions made before expiry by estimating risk, showing the relevant evidence and simulating possible product relocation.

Research question

> Can a controlled Agentic AI system improve the speed, evidence completeness and decision consistency of evaluating expiry-risk product relocations compared with a manual or rule-based workflow?

Supporting questions

1. Can the agent identify high-risk batches from imported data?
2. Can it explain recommendations using returned evidence and deterministic calculations?
3. Can it select appropriate tools and pass valid parameters?
4. Can the heatmap simulation help managers compare possible product positions?
5. Can the system prevent unauthorized state-changing actions?

3. What makes the system agentic?

The agent does more than answer questions from memory. It interprets a manager's goal, selects approved tools, retrieves scoped data, invokes deterministic calculations, explains the evidence and prepares a draft recommendation.

```
Manager asks a question
        ↓
Agent interprets the request
        ↓
Agent selects approved tools
        ↓
Tools validate scope and retrieve data
        ↓
Business logic calculates risk or compares positions
        ↓
Agent explains the evidence
        ↓
Manager approves, rejects or requests revision
        ↓
The decision is recorded in the audit log
```

Example question:

> "Which yogurt batches in Store 1 may expire within seven days, and what should I review first?"

The agent should retrieve the relevant batches, estimate demand before expiry, rank the risk, identify possible actions and present the evidence. It must not directly change quantities, create a purchase order, dispose of products or activate a new layout without approval.

4. MVP scope

Included

- Basic authentication and organization/store access
- CSV import with validation
- Product, store, batch and sales records
- Deterministic demand and expiry-risk calculations
- Dashboard with risk summary and drill-down
- Agentic natural-language decision support
- Recommendation and approval workflow
- Manager-configured 2D store heatmap
- Proposed-layout simulation
- Audit log
- Evaluation against a manual or rule-based baseline

Explicitly outside the MVP

The MVP will not include live POS integration, automatic purchasing, automatic disposal, automatic stock transfer, 3D store reconstruction, computer vision, supplier negotiation, warehouse optimization or a universal importer for every supermarket system.

5. MVP modules

Module 1: Authentication and Organization Management

Purpose: Establish the security boundary for organization, store and role access.

Main functions: Authenticate users, complete onboarding, create or join an organization with a secure join code, assign basic roles, select an authorized store and apply organization/store scope to every query.

Main input: User profile, organization or join-code choice, store selection and role information.
Main output: Authenticated session, organization ID, active store ID, role and permission checks.

Module 2: Data Import from CSV

Purpose: Integrate with an existing supermarket workflow without requiring a new POS or ERP.

Main functions: Upload, map, validate, preview and confirm CSV files. The MVP supports four logical file types: product master, store/location master, batch stock and daily sales history.

Main input: CSV file, selected file type, column mapping and import mode.
Main output: Valid and invalid row counts, error messages, imported records and an import audit record.

The importer should validate required identifiers, dates, quantities, duplicate batches and store/product references. It should not silently repair business data.

Module 3: Batch Inventory System

Purpose: Preserve expiry information that would be hidden if all units of a product were represented by one product-level quantity.

Main functions: Store batch dates and quantities, record stock movements, calculate remaining shelf life and display batches by risk.

Main input: Product, store, batch code, received date, expiry date, quantities and stock movements.
Main output: Batch list, batch details, movement history, remaining days and inputs for risk analysis.

If exact batch-level sales allocation is unavailable, the evaluation dataset will use a stated FIFO-like assumption rather than claiming perfect historical attribution.

Module 4: Dashboard

Purpose: Give managers a simple operating view without requiring an AI question for every task.

Main functions: Show selected store, high-risk batches, estimated surplus, low-stock indicators, pending recommendations, data-quality warnings and links to batch details and the heatmap.

Main input: Organization scope, store selection, batch data, sales data, risk results and recommendation status.
Main output: Filterable risk summary and navigation to the relevant evidence.

Module 5: Notification and Recommendation

Purpose: Use the agent to investigate a risk and prepare an explainable recommendation for human review.

Main functions: Identify urgent risks, call approved analysis tools, explain evidence, draft recommendations and record manager decisions.

Main input: Manager question, authorized store context, tool results and evidence IDs.
Main output: Risk explanation, recommendation draft, assumptions, approval status and audit record.

A typical workflow is: summarize the store, find expiry risks, inspect the highest-risk batch, estimate demand, find transfer candidates when relevant, create a draft and request approval.

Module 6: Store Heatmap Planner

Purpose: Let the manager configure a simple representation of the store and test possible product relocations.

Main functions: Create a grid, choose rows and columns, configure shelf levels, place multiple products on a shelf level, save layout versions and display demand, expiry and waste indicators.

Main input: Store, grid dimensions, shelf levels, capacity, storage type and product positions.
Main output: Active layout, draft layout, product positions, warnings and current-versus-proposed indicator comparisons.

The simulation checks storage compatibility and configured capacity, then recalculates indicators for the proposed position. It reports historical associations and warnings; it must not claim that moving a product will definitely increase sales.

6. Local-first technology and optional AWS migration

The required MVP can be completed without paid AWS AI services. Develop locally with Next.js, TypeScript, Tailwind CSS, Bun, Elysia, PostgreSQL, Drizzle ORM and a provider-independent agent interface. OpenCode can be used with an available free model, while a deterministic mock agent provides repeatable tests and a fallback when a free model is unavailable.

The architecture should keep the agent provider replaceable:

```
AgentProvider
├── OpenCodeAgentProvider
├── MockAgentProvider
└── Optional BedrockAgentCoreProvider
```

AWS is an optional later demonstration, not a completion dependency. If credits are available, selected components may migrate to Cognito, API Gateway, Lambda, S3, RDS PostgreSQL, Bedrock and AgentCore. The database, tool contracts, approval rules and evaluation dataset should remain the same in both modes. AWS pricing and free-tier conditions must be checked for the actual account before deployment; the proposal must not claim that Bedrock or AgentCore is automatically free.

7. Data model for the MVP

The following entities preserve the detailed data-model design required for implementation:

```
users
organizations
organization_members
stores
store_locations
products
suppliers
batches
inventory_transactions
sales_records
risk_results
notifications
recommendations
layout_versions
product_positions
approval_records
import_jobs
import_errors
audit_logs
```

The key relationships are:

```
organization → stores → locations
organization → products → batches
store + product + date → sales records
batch + forecast → risk result
risk result → recommendation → approval record
store + layout version → product positions
```

A batch stores received quantity, remaining quantity, reserved quantity, received date, expiry date and status. A risk result stores the demand estimate, days until expiry, expected sales before expiry, estimated surplus, risk level, assumptions and calculation timestamp. A recommendation stores evidence IDs, status, proposed action, assumptions and approval state. A layout version stores the grid configuration and product positions.

8. API and tool design

Application API

```
POST /imports
GET  /imports/:id
POST /batches/query
GET  /dashboard/summary
GET  /risk/expiry
GET  /notifications
GET  /recommendations
POST /recommendations/:id/approve
POST /recommendations/:id/reject
GET  /stores/:id/layouts
POST /layouts/:id/simulate
POST /layouts/:id/approve
GET  /audit-logs
```

Agent tool boundary

The agent receives only approved tools. No generic SQL tool, unrestricted shell tool or generic “update anything” endpoint is exposed.

Tool category	Purpose	Typical output
Read store and batch data	Retrieve scoped records	Current facts and timestamps
Calculate expiry risk	Compare remaining stock with expected sales before expiry	Ranked risks and evidence
Calculate demand estimate	Produce a short-horizon baseline	Estimate, method and assumptions
Find transfer candidates	Identify stores with possible demand and capacity	Candidate stores and scores
Simulate product placement	Compare current and proposed positions	Validity, warnings and indicator comparison
Draft recommendation	Convert evidence into a reviewable action	Draft recommendation and assumptions
Request manager approval	Put a draft into pending state	Approval record
Record approved decision	Store a manager’s decision	Final decision and audit record

Each tool validates organization and store scope, validates parameters, applies deterministic business rules, returns only the necessary fields and logs the call. Read-only tools may run during investigation. Drafting tools may create drafts but may not change operational stock. Approval tools require an authenticated authorized user.

Core calculations

For short-horizon demand, the MVP uses a transparent moving average or weighted moving average. Expiry risk is calculated from:

```
days_until_expiry = expiry_date - current_date
expected_sales_before_expiry = estimated_daily_demand × days_until_expiry
estimated_surplus = available_quantity - expected_sales_before_expiry
```

The result is classified into configurable levels such as expired, high, medium or low. The exact thresholds will be defined before evaluation and tested using seeded data.

9. Academic evaluation plan

Compare two workflows:

- **Baseline:** A user searches a batch table and manually interprets expiry and sales information.
- **Agentic workflow:** A user asks the agent, receives evidence-backed analysis, reviews a recommendation and approves or rejects it.

Use a controlled dataset with approximately 30–50 products, one or two stores, multiple batches, 7–30 days of sales, stock-outs, surplus batches, transfer candidates and a simple layout.

Measure task completion time, correct risk identification, tool-selection accuracy, explanation grounding, unsupported claims, unnecessary tool calls, approval-control violations, user confidence and perceived usefulness. Suggested project targets are at least 90% agreement for deterministic risk calculations, at least 80% correct first-attempt tool selection and zero unauthorized state-changing actions in test scenarios. These are FYP targets, not industry benchmarks.

10. One-year roadmap

- **Months 1–2:** Confirm scope, research question, data dictionary, CSV format and evaluation dataset.
- **Months 3–4:** Build local authentication, organization/store scope, CSV import and core records.
- **Months 5–6:** Implement sales queries, demand estimate, expiry-risk calculation, dashboard and audit logging.
- **Months 7–8:** Implement the provider interface, OpenCode adapter, mock fallback, approved tools, recommendation drafting and approval.
- **Months 9–10:** Build the configurable heatmap and layout simulation.
- **Month 11:** Run deterministic tests, user comparison tasks and optional AWS demonstration.
- **Month 12:** Analyze results, document limitations and prepare the dissertation and final demonstration.

11. Main risks and controls

- **Incorrect recommendation:** Use deterministic tools for quantities and risk; require evidence and assumptions.
- **Unauthorized operation:** Separate read-only tools from approval tools and enforce authorization in the backend.
- **Scope expansion:** Keep CSV as the integration boundary and exclude live POS, procurement and warehouse features.

- **Insufficient data:** Use synthetic or anonymized data and distinguish demonstration results from real operational validation.
- **Free-model or AWS availability:** Keep the mock agent and local workflow as the required fallback.
- **Heatmap becoming a visual gimmick:** Connect it to risk, demand, waste and stock-out indicators and evaluate whether it changes a planning decision.

12. Final project definition

Recommended title:

> ShelfFlow: A Local-First Agentic AI Decision Support System for Supermarket Expiry-Risk and Store-Layout Planning

MVP statement:

> ShelfFlow upgrades an existing supermarket stock workflow. Managers import product, batch, sales and store-layout data from CSV files. Deterministic services calculate expiry risk and demand indicators, while an Agentic AI uses controlled tools to investigate risks, simulate possible product positions and prepare evidence-backed recommendations. Managers retain final approval, and the system records decisions without automatically executing irreversible operational actions. The MVP runs locally and can optionally migrate selected components to AWS when funding is available.

Appendix A: Core tools and page mapping

Core tools

```
get-store-summary
get-expiry-risks
get-batch-detail
calculate-demand-estimate
find-transfer-candidates
simulate-product-placement
create-recommendation-draft
record-manager-decision
```

Main pages

Page	Route	Main responsibility
Login	`/login`	Authenticate and route first-time users
Onboarding	`/onboarding`	Complete profile and create or join an organization
Dashboard	`/dashboard`	Show store summary, urgent risks and recommendations
CSV import	`/imports`	Upload, validate and confirm data
Batch list/detail	`/inventory/batches`	Filter batches and inspect risk evidence
Recommendations	`/recommendations`	Review drafts and approval status
Agent assistant	`/assistant`	Ask questions and view evidence-backed responses
Store heatmap	`/stores/:storeId/layout`	Configure grid and product positions
Layout simulation	`/stores/:storeId/layout/simulations/:id`	Compare current and proposed positions
Organization settings	`/settings/organization`	Manage stores, roles and join code

Page	Route	Main responsibility
Audit log	`/settings/audit-log`	Review imports, approvals and layout activations

Simplified user flow

```
/login
  ├── first-time user → /onboarding → /dashboard
  └── returning user → /dashboard

/dashboard
  ├── urgent risk → /inventory/batches/:batchId
  ├── recommendation → /recommendations/:recommendationId
  ├── import data → /imports
  └── plan layout → /stores/:storeId/layout
```

References